

Smart Water Resource Management

Data Engineering, Real-Time Monitoring and Drift Detection Report

Project scope: Build a Python data-engineering layer that receives AI-model predictions over TCP, reconstructs records, validates them, separates clean and bad data, stores outputs, generates 20-second operational reports, and detects model behavior drift.

Important: The three AI models are treated as existing prediction services. This project does not train or fine-tune the models; it monitors their incoming outputs.

1. System Architecture

TCP Stream → Receive → Buffer/Framing → JSON Parsing → Validation → Clean/Bad Storage → Real-Time Reporting → Drift Detection → Analytical Report

Because TCP does not preserve application-level message boundaries, the receiver maintains a text buffer and splits complete records on newline characters. Incomplete trailing data is kept in the buffer until the next TCP packet completes it. JSON parsing errors are handled without terminating the receiver.

2. Validation and Data Quality

Each record is checked for `reading_id`, `station_id`, `model_name`, model-specific `predicted_label`, `confidence_score`, `response_time_ms` and `timestamp`. Invalid records are written separately with the validation errors that caused rejection.

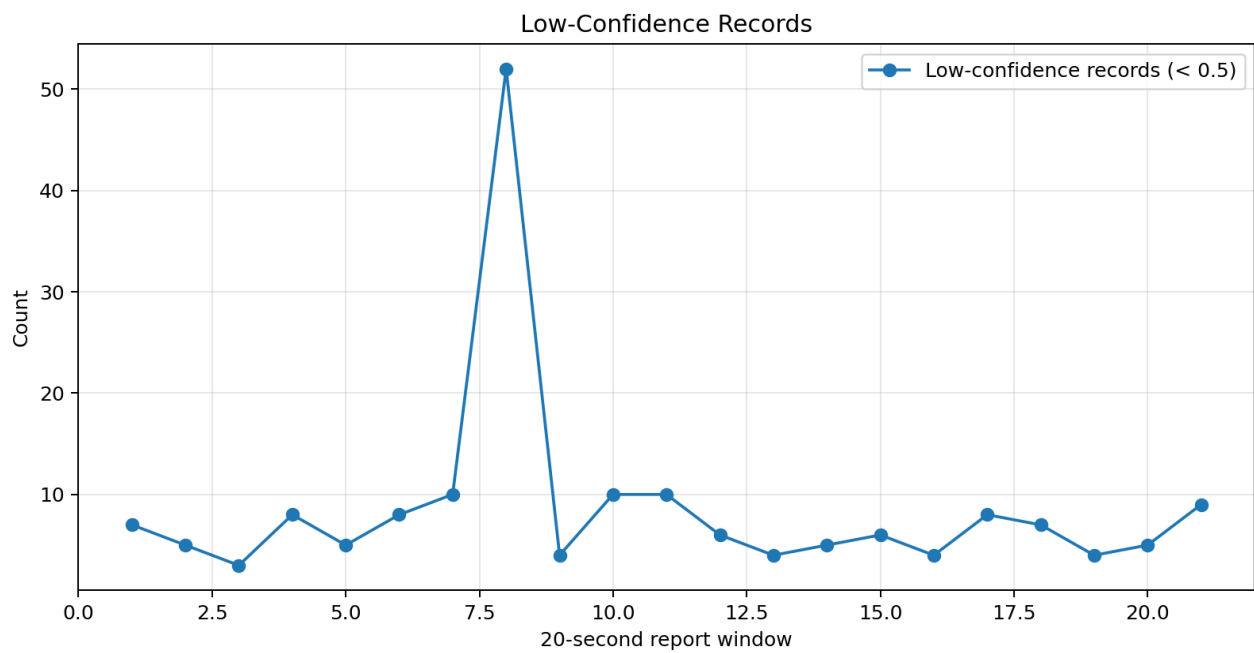
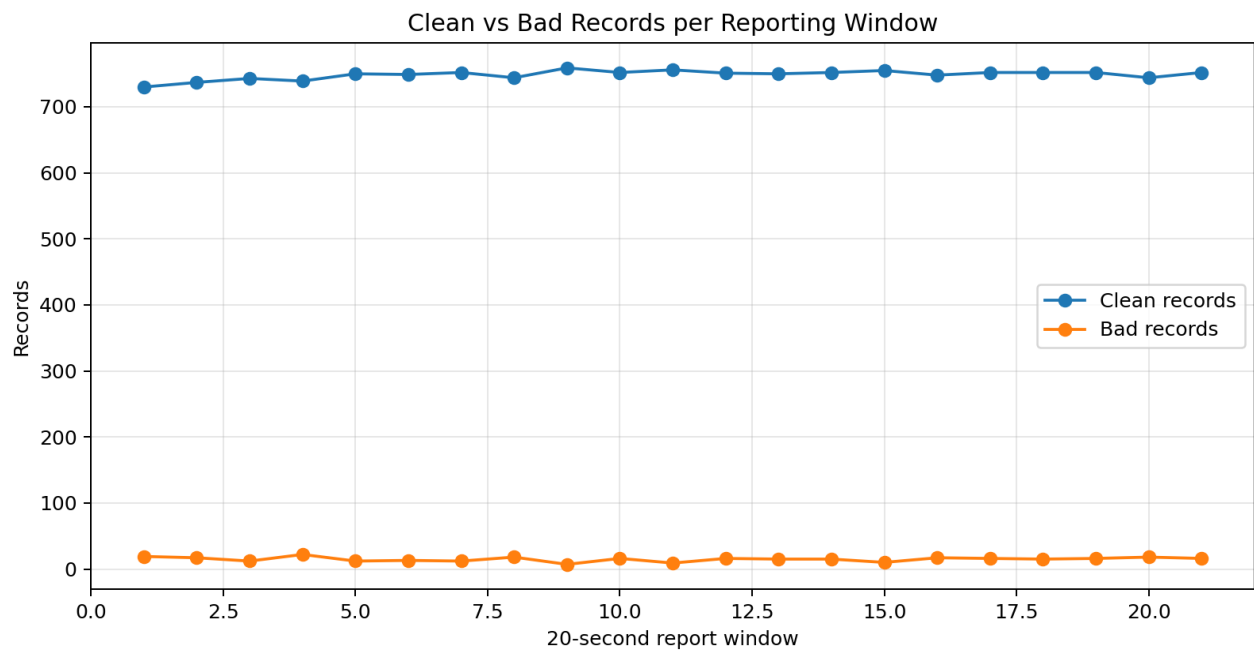
Metric	Value
Clean records	15,847
Bad records	314
Total records	16,161
Observed bad percentage	1.94%
Active stations	10
20-second reports	21
Drift alerts	3

Validation error	Occurrences
invalid confidence_score	314
invalid response_time_ms	314
invalid predicted_label	161
invalid model_name	115
invalid reading_id	103
invalid station_id	72
invalid timestamp	63

The sum of validation-error occurrences is greater than the number of bad records because one record can contain multiple simultaneous validation failures.

3. Real-Time Reporting

A report row is appended every 20 seconds. It includes record counts, model-level average confidence, model-level average response time, low-confidence count and active station count. The final execution produced 21 complete reporting windows.

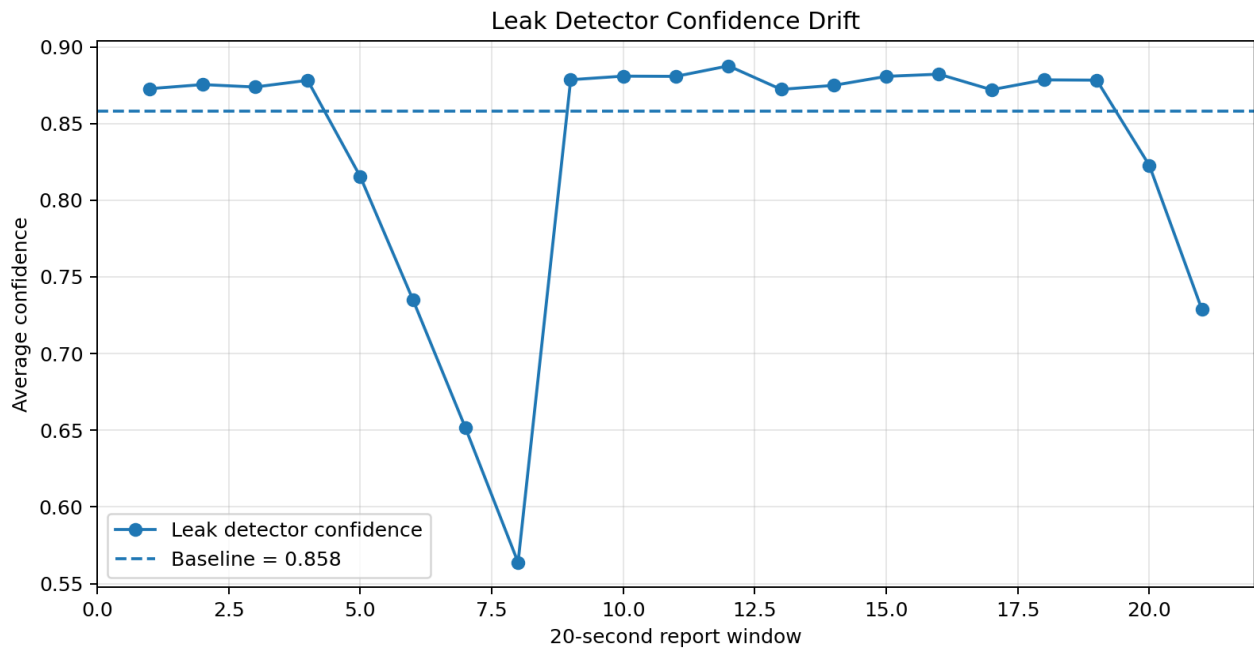


4. Drift Detection Method

Drift is detected without machine learning by comparing a fixed baseline with independent complete windows. The implemented detector uses a 60-record baseline, 20-record windows, a 15% deviation threshold, and a persistence requirement of 3 consecutive windows.

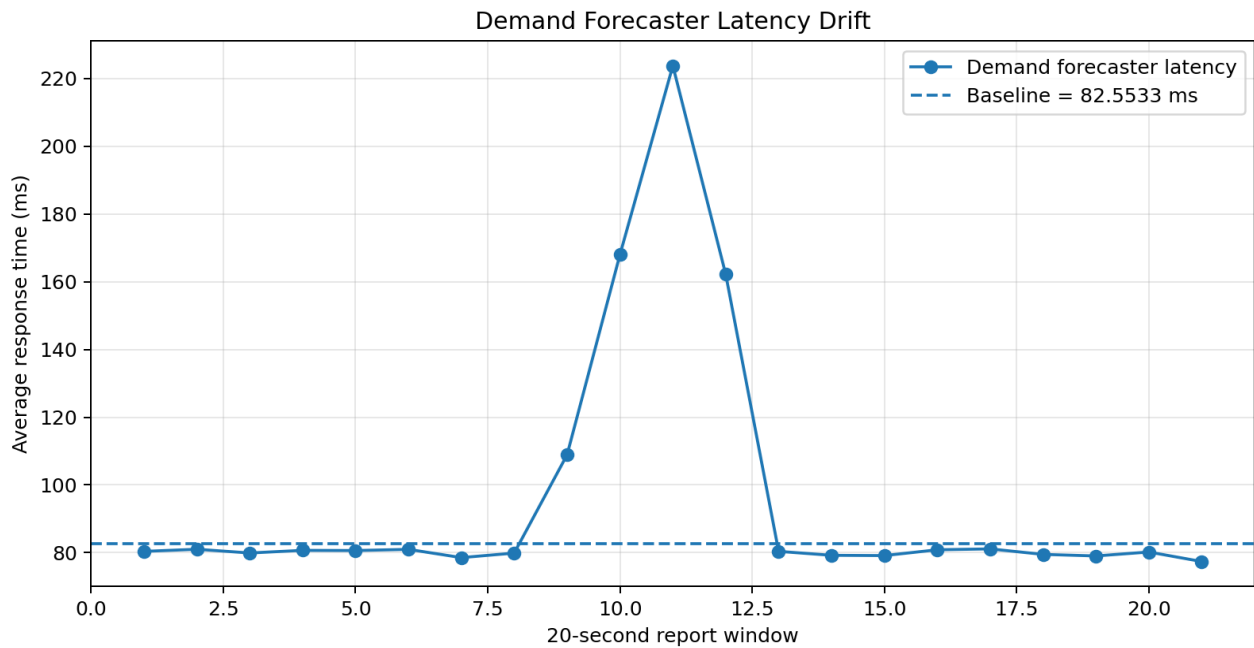
Leak detector: confidence_score is monitored and downward deviation is considered drift. **Demand forecaster:** response_time_ms is monitored and upward deviation is considered drift. **Pressure-drop predictor:** no drift rule was defined in this implementation because the project requirements did not specify a metric for it; it is still monitored in the reports.

5. Observed Leak-Detector Drift



The confidence remains around 0.87–0.88 during normal operation, then falls sharply across windows 5–8, reaching 0.5639. This is consistent with a persistent downward behavioral change, rather than an isolated noisy observation. The detector therefore generated a drift alert at 22:06:01: baseline 0.858 versus window average 0.6887, a 19.73% deviation across 3 consecutive windows.

6. Observed Demand-Forecaster Drift



Demand-forecaster latency is near 80 ms during normal operation, then increases to 108.89 ms, 168.03 ms and 223.80 ms in successive windows. The detector subsequently records a latency-drift alert at 22:06:51: baseline 82.5533 ms versus window average 105.125 ms, a 27.34% deviation across 3 consecutive windows. The later windows return close to the normal latency range, showing that the system can observe both the onset and recovery of the injected behavioral change.

Timestamp	Model	Type	Baseline	Window avg.	Deviation	Consecutive
2026-09-09 22:06:01	leak_detector	confidence	0.858	0.6887	19.73%	3
2026-09-09 22:06:51	demand_forecaster	latency	82.5533	105.125	27.34%	3
2026-09-09 22:11:00	leak_detector	confidence	0.858	0.7063	17.69%	3

7. Analytical Questions

Q1 — Can the same Python architecture handle 50,000 records/sec?

Python can remain the implementation language, but the current synchronous per-record CSV-writing design is not suitable for 50,000 records/sec. At that scale, ingestion should be separated from processing using a durable message broker, multiple consumers, batching and a scalable storage layer. The current project demonstrates the processing logic, not production-scale throughput.

Q2 — How should multiple streams be handled?

The ingestion layer should accept multiple producers and decouple them from downstream processing. A broker or queue can partition traffic and allow parallel consumers. The current local server is adequate for the challenge but should not be interpreted as the final production architecture.

Q3 — Is zero data loss guaranteed?

No. The current challenge implementation does not provide an end-to-end zero-loss guarantee. A production design would require durable buffering, acknowledgements, retries, checkpointing and idempotent processing so that records can be recovered after failures.

Q4 — How can noise be separated from real drift?

A single abnormal observation should not trigger drift. The implemented approach establishes a baseline and evaluates complete windows relative to it, requiring the deviation to persist for three consecutive windows. For a

production system, station-specific and time-of-day baselines plus EWMA, CUSUM or change-point methods could improve sensitivity.

8. Ambiguities, Assumptions and Limitations

The project specification did not prescribe an exact drift algorithm, threshold, baseline size or persistence rule. Therefore, this implementation explicitly assumes 60 baseline records, 20-record windows, 15% deviation and 3 consecutive windows. The specification also does not define a drift metric for every model, so only leak-detector confidence and demand-forecaster latency are used for automated drift alerts. Finally, only complete 20-second report windows are written; a final partial window is not treated as a normal report. These assumptions should be documented rather than hidden.

9. Testing and Execution

The drift detector test suite contains 10 tests covering history, averages, baseline creation, fixed baselines, persistence, no-drift behavior, latency drift and the pressure model's no-rule case. The complete stream was also executed for more than six minutes and produced real CSV outputs and drift alerts.

10. Final Conclusion

The implemented pipeline successfully demonstrates the requested data-engineering workflow: TCP framing, JSON parsing, validation, clean/bad separation, persistent CSV storage, periodic reporting and non-ML drift detection. The final real execution produced 16,161 records, of which 15,847 were clean and 314 were bad. All 10 stations were active. Three drift alerts were generated, corresponding to the injected leak-detector confidence change and demand-forecaster latency change. The main production limitations are throughput, durability and scalable storage; these are architectural considerations for deployment beyond the challenge environment.